

面向 CEC 场景的 LLM 推理优化：算法 研究与应用综述

Cloud-Edge-Client Collaborative LLM Inference Survey

研究综述报告

云-边-端协同计算与大模型推理优化

版本： v1.0

日期： 2026 年 5 月

摘要

大语言模型推理正在从云端集中式服务走向端、边、云协同参与的分布式服务形态。随着端侧设备、接入边缘、区域边缘和中心云共同进入推理路径，系统瓶颈不再只是单个模型或单个 GPU 实例的执行效率，而是模型部署、请求调度、KV/上下文状态管理、服务路径选择和业务 QoS 之间的联合权衡。CEC 场景下的 LLM inference 因而不能简单理解为云侧 serving 能力的边缘下沉，而需要在异构资源、动态网络、用户移动、隐私边界和服务连续性约束下重新组织推理系统。

本文围绕 CEC 场景下的大语言模型推理优化展开综述。首先，本文梳理 LLM inference 的基本执行机制、问题空间和以优化对象为主轴的 Layer-Scope 分类框架，并分析 CEC 相较云侧集中式 serving 带来的优化重心迁移。在此基础上，本文重点讨论三类算法问题：其一是资源自适应的模型切分部署方法，包括 layer-wise、tensor/operator-level、expert/component-level、inference phase 以及 semantic/token-level partitioning；其二是业务感知的 batching 参数优化方法，包括 queue-level dynamic/admission batching、continuous/iteration-level batching、chunked prefill/blended batching 和 disaggregated prefill-decode batching；其三是 LLM inference task offloading，包括 request/session routing、multi-model/multi-agent workflow orchestration 以及 context/KV/runtime state migration and offloading。进一步地，本文总结三类算法之间的协同关系，讨论其在华为 Ascend 推理栈、vLLM Ascend、MindIE 和 CANN 等现有基础设施上的工程落点，并给出面向验证环境的实施路线、实验指标、baseline 设计和未来研究方向。本文旨在为云-边-端协同大模型推理系统的算法选型、实验验证和工程落地提供系统化参考。

目录

一、引言	5
1.1 研究背景与应用动机	5
1.2 CEC 场景下边缘 AI 推理的关键挑战	5
1.3 国内外研究现状概述	6
1.4 本文研究范围与主要贡献	7
二、LLM 推理基础、问题空间与优化分类	9
2.1 LLM 推理的基本执行机制	9
2.2 LLM 推理的问题空间：从执行机制到系统瓶颈	12
2.3 LLM Inference Optimization Taxonomy：以优化对象为主轴的分类	14
2.4 工业级 LLM Inference Framework 代表性机制映射	17
2.5 小结	19
三、CEC 场景下的大模型推理	20
3.1 CEC LLM 推理的系统形态	20
3.2 CEC 相较云侧集中式 LLM Serving 新增的问题空间	21
3.3 CEC 场景下 LLM Inference 优化重心的转移	21
3.4 本文采用的 CEC LLM Inference 综述框架	22
3.5 小结	24
四、CEC 下的资源自适应的模型切分部署方法	25
4.1 Layer-wise / Vertical Partitioning	29
4.2 Tensor / Operator-level Partitioning	30
4.3 Expert / Component-level Partitioning	32
4.4 Inference Phase Partitioning	33
4.5 Semantic / Token-level Partitioning	35
4.6 算法方案对比与面向华为验证环境的建议	36
4.7 小结	38
五、CEC 下的业务感知 Batching 参数优化方法	40
5.1 Queue-level Dynamic / Admission Batching	43

5.2 Continuous Batching / Iteration-level Batching	45
5.3 Chunked Prefill / Blended Batching	46
5.4 Disaggregated Prefill-Decode Batching	48
5.5 算法方案对比与面向华为验证环境的建议	49
5.6 小结	51
六、CEC 中的 LLM Inference Task Offloading	53
6.1 Request / Session-level Routing and Offloading	54
6.2 Multi-model / Multi-agent Workflow Orchestration	56
6.3 Context / KV / Runtime State Offloading and Migration	57
6.4 面向项目场景的长期成本感知请求卸载方案	59
6.5 小结	62
七、三类算法的协同优化框架	65
7.1 三类算法的基本耦合关系	65
7.2 模型/服务部署与请求卸载的双时间尺度协同	65
7.3 模型切分、阶段解耦与 batching/scheduling 的协同	66
7.4 请求/工作流路由与 runtime state 管理的协同	66
7.5 多用户移动场景下的跨层闭环	67
7.6 控制面、数据面与状态面的工程实现	68
7.7 小结	68
八、面向 vLLM-Ascend 的 CEC 推理工程实现方案	69
8.1 工程定位	69
8.2 总体架构	69
8.3 关键扩展模块	70
8.4 既有基础与可继承能力	71
8.5 小结	71
九、实验评估与对比指标	73
9.1 实验场景	73
9.2 Baseline 设置	73
9.3 指标体系	74
9.4 消融实验	75

9.5 预期收益分析	76
十、挑战与未来方向	77
10.1 异构资源与性能建模	77
10.2 状态迁移与一致性	77
10.3 在线策略稳定性	78
10.4 隐私与合规	78
10.5 未来研究方向	78
十一、总结	80
参考文献	81

一、引言

1.1 研究背景与应用动机

大语言模型推理正在从云端集中式服务，走向端侧、边缘和云端共同参与的协同服务形态。过去几年中，LLM inference 优化大体形成了两条成熟路线：一条是 on-device inference，通过压缩、量化、小模型和端侧 NPU/GPU 加速，使模型尽可能靠近用户运行；另一条是 cloud / datacenter LLM serving，通过高性能 kernel、continuous batching、KV Cache 管理、模型并行和多实例调度，在云侧 GPU 集群中提高吞吐、降低尾延迟并摊薄服务成本。已有 LLM serving 系统与综述工作分别从 runtime、memory management、batching/scheduling 和 distributed serving 等角度总结了这些机制 [4]-[8], [73]-[75]。

这两条路线分别解决了局部问题，却都难以单独覆盖真实业务中的全部约束。端侧推理具有低时延和数据本地性优势，但受算力、内存、功耗和散热限制，很难长期承载复杂大模型和高并发服务。云侧推理资源充足、框架成熟，却必须经过网络访问，端到端时延、带宽消耗、隐私风险和服务路径依赖都会进入业务关键路径。与此同时，接入边缘、区域边缘和近用户算力节点中存在大量分散、异构且动态变化的资源，它们既无法简单等同于端侧设备，也不同于数据中心中的同构 GPU 集群。

CEC (Collaborative Edge Computing, 协作边缘计算) 正是在这种背景下成为 LLM inference 的重要系统形态。如图 1 所示，CEC 将泛在、地理分布的端侧与边缘设备组织为可协作的资源池，并通过 Wi-Fi 接入点、路侧单元、5G 基站和网关等基础设施连接中心云与多类终端，共同完成计算任务。CEC 并不是简单地把模型从云端搬到边缘，而是把端侧、边缘和中心云视为一个多层协同资源池，使推理请求能够根据模型规模、请求复杂度、网络状态、节点负载、隐私要求和 QoS 目标动态选择模型形态、执行位置和服务路径。端-边-云协同智能研究已经指出，多层系统的关键在于资源、链路和服务位置之间的协同，而不是单点计算能力的替换 [13]。因此，CEC LLM inference 的核心问题不再只是“如何让模型跑得更快”，而是如何在异构、分布式、动态变化的端-边-云环境中，以可控成本满足业务时延、吞吐、服务连续性和数据隐私要求。

1.2 CEC 场景下边缘 AI 推理的关键挑战

CEC 场景下的边缘 AI 推理首先受到资源异构和碎片化的约束。端侧设备、接入边缘、区域边缘和中心云在 CPU、GPU、NPU、内存、显存、功耗预算和加速库支持上差异显著，模型部署不能沿用中心云中“节点规格相近、互联稳定、控制面统一”的假设。同一个模型在不同硬件上可能面临完全不同的显存瓶颈、算子支持和通信代价，部署策略也必须随资源状态动态调整。

LLM 推理本身还具有显著的状态和内存压力。Prefill 阶段会为后续 decode 生成 KV Cache，de-

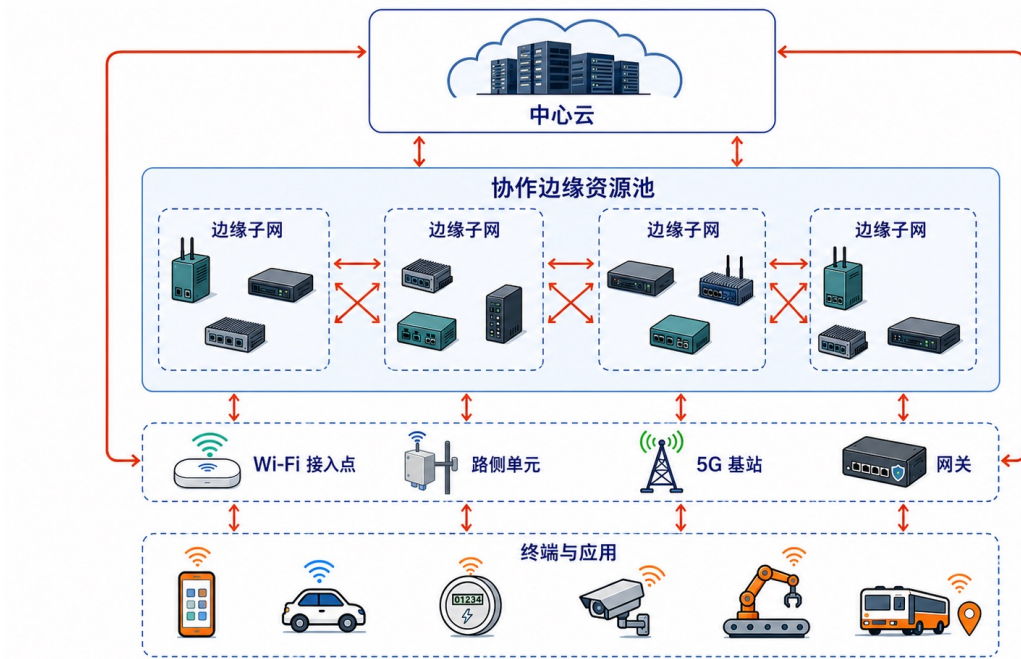


图1 协作边缘计算示意图。协作边缘计算汇聚泛在、地理分布设备的计算资源以共同执行计算任务，从而扩展可用资源池，实现低时延数据处理、灵活设备接入与更广的服务覆盖。

code 阶段又会随着输出长度不断追加状态；在长上下文、多轮会话和高并发场景中，KV Cache、prefix cache 和 session state 往往成为比单次计算更难管理的运行时资源。一旦这些状态跨节点访问、迁移或重算，通信成本、服务连续性和隐私边界都会同时受到影响。

请求动态性进一步放大了调度难度。用户并发数、prompt 长度、输出长度、请求复杂度和 SLA 时延要求都在持续变化，边缘节点又常常缺乏云侧大规模请求池，静态 batching、固定 batch size 和固定服务路径难以稳定满足业务目标。对于移动用户，接入点和最优服务实例还会随位置变化而变化，最近节点不一定是负载最合适或端到端时延最低的节点；若用户会话中的 KV 或上下文状态仍保留在旧边缘节点，服务切换还可能引发状态迁移、远程访问或重新 prefill。

因此，CEC LLM inference 的挑战并不是单一技术点，而是模型部署、batching 调度、task offloading、状态管理和路由之间的耦合。模型切分、prefill/decode 分离、KV 迁移、跨节点执行和多入口路由都可能带来新的 activation、hidden state、KV Cache 或 session state 传输成本。系统需要在计算收益和通信代价之间持续权衡，并在 QoS、隐私和服务连续性之间给出可解释的在线决策。

1.3 国内外研究现状概述

现有研究为 CEC LLM inference 提供了多方面基础。一方面，on-device inference 关注模型压缩、低比特执行、小模型设计和端侧硬件加速，使部分请求能够在用户设备本地完成；FlexGen、Petals、HexGen 等工作则进一步展示了资源受限或异构环境下的大模型推理可以通过分层存储、协作式 block 放置和异构并行来突破单节点容量限制 [1], [2], [3]。另一方面，cloud/datacenter LLM serving 已经围绕

continuous batching、PagedAttention、chunked prefill、prefill/decode disaggregation、KV cache management 和多实例调度形成了较成熟的系统路线, Orca、vLLM、Sarathi-Serve、DistServe、SGLang 和 Mooncake 等系统分别代表了不同层面的关键机制 [4]-[8], [21]。

边缘计算领域长期研究 task offloading、资源分配、移动性管理和网络感知调度, 但传统 MEC/CEC 任务通常被抽象为给定输入大小、计算量、deadline 和能耗约束的 job。LLM inference 改变了这一抽象: 推理时间取决于 prompt、输出长度、decode step、batch 状态和 KV 状态, 复杂请求还可能涉及多模型、多 agent 或多阶段执行。因此, CEC LLM inference 位于 LLM serving systems 与 CEC task offloading 的交叉处, 既要继承 LLM serving 中的 prefill/decode、KV Cache、batching、speculative decoding 和分布式执行问题, 又要处理端-边-云资源异构、动态网络、用户移动、隐私约束和服务连续性。

基于这一交叉特征, 本文将相关工作组织为三条主线。第一条是 **CEC 下的资源自适应模型部署方法**, 关注模型结构、模型组件、执行阶段和生成职责如何在端、边、云之间切分、放置和重配置; 这一方向直接对应后文的 layer-wise / tensor-level / expert-level / phase-level / semantic-token-level 等切分范式。第二条是 **CEC 中的业务感知 batching 参数优化算法**, 关注请求进入推理实例后如何根据 SLO、队列状态、KV 占用和 prefill/decode 干扰选择 batch 形成与更新策略。第三条是 **CEC 中的 LLM inference task offloading**, 关注完整请求、会话、multi-agent workflow 以及 context/KV/runtime state 如何在端、边、云之间路由、卸载、迁移和复用。这三条主线分别从空间部署、时间调度和服务路径三个角度刻画 CEC LLM inference 的核心优化问题。

1.4 本文研究范围与主要贡献

本文聚焦 **CEC 场景下的大语言模型推理优化问题**。研究范围限定在推理阶段, 不讨论大模型训练、预训练或微调过程; 重点关注端侧、接入边缘、区域边缘和中心云之间的大模型推理协同, 包括模型动态切分、自适应 batching、prefill/decode 调度、分层 KV/状态管理、任务 offloading、策略路由、移动性感知请求调度和服务连续性保障。

对于底层 kernel 优化、硬件实现和模型结构设计, 本文仅在其影响 CEC 推理系统设计时进行讨论。例如, 低比特 kernel 会影响模型能否部署在端侧或边缘设备上; KV Cache layout 会影响状态迁移和缓存复用; partition-friendly model design 会影响模型是否适合跨端-边-云切分执行。本文的重点不是单独评估某个 kernel 或模型结构, 而是从 CEC 系统角度分析这些技术如何影响大模型推理的部署、调度、状态管理和服务策略。

本文首先从 prefill/decode、KV Cache、batching/scheduling、decoding runtime 和主要服务指标出发, 梳理 LLM inference 的基本执行机制, 并进一步解释模型规模、KV 状态、请求动态性、decode 串行性和多实例协同为何会成为推理系统的核心瓶颈。在此基础上, 本文构建面向优化对象的 Layer-Scope taxonomy, 将 LLM inference optimization 划分为 Model-Level、Kernel-Level、Runtime-Level 和

Serving Policy & Routing-Level, 并以 Local / Distributed 作为执行范围副轴, 用于分析不同方法的 primary optimization object、execution scope 和 secondary effects。

随后, 本文分析 CEC 环境如何改变 LLM inference 的优化重心。相较云侧集中式 serving, CEC 场景需要额外处理端-边-云资源异构、边缘低并发、动态网络、用户移动、状态迁移、隐私边界和 QoS 约束, 因此各优化层都需要从单点效率转向跨节点、跨状态、跨服务路径的协同决策。围绕这一变化, 本文提出三类面向 CEC 的综述主线: 资源自适应模型部署方法、业务感知 batching 参数优化算法和 LLM inference task offloading, 并分别总结其核心问题、代表工作、决策变量、优化目标和面向验证环境的算法建议。最后, 本文从时延、吞吐、QoS、资源利用率、通信开销、状态迁移成本、隐私约束、服务连续性和系统可部署性等角度归纳评估维度, 并讨论跨层协同优化、状态感知 routing、移动性预测、异构 runtime、分层模型服务和真实系统落地等未来挑战。

二、LLM 推理基础、问题空间与优化分类

2.1 LLM 推理的基本执行机制

LLM 推理可以理解为一个在线生成服务过程：系统接收用户输入上下文，并以自回归方式逐 token 生成输出。与传统 DNN 推理相比，LLM 推理的特殊性在于输入长度和输出长度动态变化，decode 阶段存在强状态依赖，KV Cache 会随上下文长度和生成长度持续增长，系统需要在吞吐、时延、显存占用和服务质量之间持续权衡。如图 2 所示，serving policy 是统筹请求准入、路由、排序、排队和执行组织的控制面；其中的 batching/scheduling 模块根据 SLO、优先级、请求长度、实例负载和 KV 容量等状态调整 batching window、token budget、prefill/decode quota 和 active set，形成待执行的活动批次。活动批次中既可以包含尚待处理输入上下文的 prefill tasks，也可以包含正在逐 token 迭代的 decode tasks，调度器分别将两类工作下发到对应执行路径。底层 inference runtime 随后执行 Prefill 和 Decode：Prefill 并行处理输入上下文并生成初始 KV Cache 和首 token；Decode 在每轮迭代中读取历史 KV、写入新增 KV，并将生成结果持续流式返回。运行时再将 TTFT、TPOT/ITL、队列长度和 KV 使用情况反馈给 serving policy，支持完成请求退出和新请求进入。

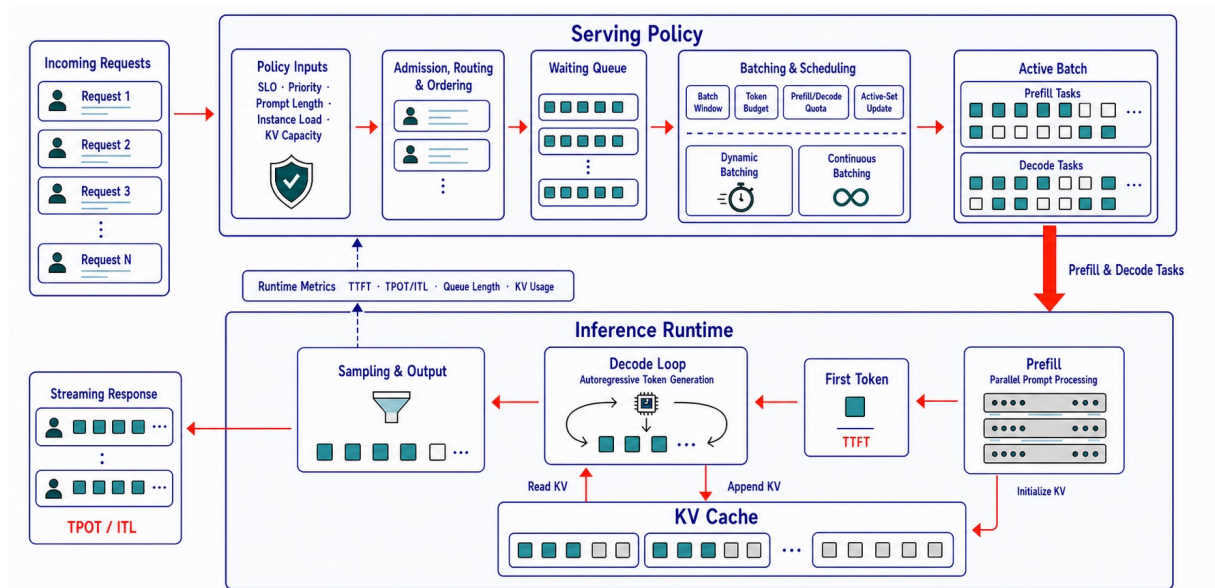


图 2 LLM serving 基础服务流程。Serving policy 作为上层控制面，内部包含准入、路由、排序、请求队列和 batching/scheduling，并根据业务与运行状态形成同时包含 prefill tasks 与 decode tasks 的活动批次；inference runtime 分别执行 Prefill、KV Cache 管理和逐 token Decode，最终形成流式响应并将运行指标反馈给 serving policy。

2.1.1 Prefill 与 Decode

LLM 推理通常分为两个阶段：**Prefill** 和 **Decode**。

Prefill 阶段负责处理完整输入上下文，包括用户 prompt、历史对话、系统提示词和检索增强内

容等。模型会对整段上下文并行计算 attention 和 FFN，并为每一层生成后续 decode 需要复用的初始 KV Cache。该阶段矩阵规模较大、并行度较高，通常更偏 compute-bound，主要影响 TTFT，即 Time To First Token。

Decode 阶段负责逐 token 生成输出。每一步 decode 输入上一步生成的新 token，同时读取历史 KV Cache，计算当前 token 的 attention 和 FFN，并将当前 token 对应的新 KV 写回缓存。该阶段每步计算粒度较小、状态依赖强、访存频繁，通常更偏 memory-bound 或 latency-bound，主要影响 TPOT、ITL 和持续生成吞吐。

Prefill 和 Decode 的资源特征差异是 LLM serving 优化的核心背景之一。Prefill 更像大矩阵并行计算，Decode 更像高频、小粒度、状态密集的在线执行。因此，很多 serving 系统会围绕 prefill/decode 的差异设计 batching、调度、KV 管理和分布式执行策略。

2.1.2 KV Cache

KV Cache 是 LLM decode 阶段最核心的运行时状态。自回归生成中，第 t 个 token 的 attention 需要访问前面所有 token 的 key/value。如果每一步都重新计算历史 token 的 key/value，会产生大量重复计算。KV Cache 的作用是在 prefill 阶段保存历史 token 的 key/value，在 decode 阶段复用这些历史 KV，只为新 token 计算新增 KV。

KV Cache 降低了 decode 阶段的重复计算，但也引入了显著的内存状态问题。KV 占用会随 batch size、上下文长度、输出长度、模型层数和 KV head 数增长。在长上下文、多轮对话和高并发场景中，KV Cache 往往成为显存容量和显存带宽的重要瓶颈。vLLM 的 PagedAttention、vAttention 的动态 KV memory management、Mooncake 的 KVCache-centric serving，以及近期 inference systems survey 都将 memory management 作为 LLM inference systems 的关键主题，覆盖 paged memory、eviction、offloading、quantization 和 cache persistence 等方向 [5], [20], [21], [73], [75]。

典型 KV 相关优化如表 1 所示：

表 1 典型 KV Cache 相关优化及作用

方向	作用
PagedAttention / memory paging	降低 KV 分配碎片，提高显存利用率
Prefix cache / RadixAttention	复用共享前缀，减少重复 prefill
KV compression / quantization	降低 KV 存储和传输成本
KV eviction	在显存不足时淘汰低价值 KV
KV offloading	将部分 KV 放到 CPU、本地存储、远端内存或其他节点
Distributed KV cache	在多 worker / 多节点场景下管理 KV 状态分布

2.1.3 Batching 与 Scheduling

Batching 是 LLM serving 提高硬件利用率和吞吐量的核心机制。单个请求，尤其是 decode 阶段，每一步计算粒度较小。如果只服务单请求，加速器容易出现算力空闲。Batching 通过把多个请求合并执行，扩大矩阵运算规模，提高 GPU/NPU occupancy，并摊薄 kernel launch、调度和访存开销。

但 LLM batching 与传统 DNN batching 不同。传统推理服务系统中的 dynamic batcher 通常在请求执行前按等待窗口和 batch size 形成批次 [10]；LLM 请求则具有输入长度、到达时间和输出长度差异大，decode 阶段请求逐步完成，batch slot 动态空出，长 prompt prefill 阻塞短请求 decode 等特征。因此，现代 LLM serving 系统通常需要 dynamic batching、continuous batching、iteration-level scheduling 和 chunked prefill 等机制。Orca 和 Sarathi-Serve 分别代表 iteration-level scheduling 与 chunked prefill 两条典型路线，近期 inference-system survey 也将 batching 和 scheduling 与 kernel design、memory management 一起作为 LLM inference systems 的核心组成部分 [4], [6], [73]-[75]。

典型调度机制如表 2 所示：

表 2 典型 LLM Serving 调度机制及作用

机制	作用
Dynamic batching	根据请求到达情况动态组成 batch
Continuous batching	在每个 decode iteration 动态加入新请求、移除完成请求
Iteration-level scheduling	以 token iteration 为粒度调度请求
Chunked prefill	将长 prompt prefill 拆成多个 chunk，降低对 decode 的阻塞
Priority / fairness scheduling	根据 SLA、优先级和公平性调整请求顺序
Admission control	在资源紧张时控制进入系统的请求量

Batching 的核心目标不是简单最大化 batch size，而是在 throughput、TTFT、TPOT、ITL、显存占用和 SLA 达标率之间做动态平衡。

2.1.4 主要性能指标

LLM inference 通常同时关注表 3 中的指标：

表 3 LLM 推理主要性能指标

指标	含义
TTFT	Time To First Token，首 token 延迟
TPOT	Time Per Output Token，平均每个输出 token 的生成时间
ITL	Inter-token Latency，相邻 token 输出间隔
E2E latency	从请求到完整响应结束的端到端时延
Throughput	单位时间处理的 request 数或 token 数
Goodput	满足 SLA 的有效吞吐
GPU/NPU utilization	加速器计算单元利用率
Memory footprint	权重、activation、KV Cache 等占用
Cost per token	单位 token 的计算、显存和服务成本

2.2 LLM 推理的问题空间：从执行机制到系统瓶颈

LLM 推理的系统瓶颈与其执行机制密切相关。大规模模型参数和复杂的 attention、FFN 计算带来较高的算力与显存开销；Prefill 需要并行处理完整输入上下文，而 Decode 受自回归依赖限制，只能逐 token 生成，二者不同的资源特征容易造成计算效率和时延瓶颈；KV Cache 随上下文长度和并发请求持续增长，又会增加显存容量、访存带宽和状态管理压力。同时，请求到达时间、输入长度和输出长度具有动态性，使 batching 与 scheduling 更加复杂；当推理扩展到多 GPU、多实例或多节点时，还会进一步引入通信、状态传输和服务编排开销。基于这些由执行机制产生的问题，本节从计算复杂度与模型规模、KV Cache 与内存状态、请求动态性与调度、Decode 串行性与生成加速，以及多实例、多 GPU 与服务级编排五个方面分析 LLM 推理的主要系统瓶颈。

2.2.1 计算复杂度与模型规模问题

LLM 参数规模大，单次 forward 的 FLOPs 高，attention 和 FFN 都会带来大量矩阵计算。长上下文进一步放大 attention 的计算和存储开销。相关优化通常关注如何从模型结构、参数表示或算子执行上降低计算复杂度。

这一类瓶颈首先体现为权重加载和常驻显存成本高，其次体现为 attention 与 FFN 带来的高计算量。长上下文会进一步推高 attention 计算和 KV 存储压力；MoE 等条件计算结构虽然降低了单 token 激活参数量，却引入 expert routing、expert placement 和 all-to-all 通信等新的系统问题。因此，相关优化通常沿着两条路径展开：一是通过量化、剪枝、蒸馏、低秩分解等方法降低模型参数和计算量；二是通过 GQA/MQA/MLA、efficient attention、MoE 优化、低比特执行和高效 attention kernel 改善模型结构或执行路径。

2.2.2 KV Cache 与内存状态问题

KV Cache 将 decode 从重复计算历史上下文变为增量计算当前 token，但代价是引入持续增长的运行时状态。上下文越长、batch 越大、并发越高，KV Cache 越容易成为显存容量和带宽瓶颈。

KV 问题的难点在于它不是静态模型权重，而是随请求生命周期动态增长和释放的状态。KV Cache 的显存占用会随上下文长度和并发度增长，动态分配释放又容易产生碎片；在多轮对话和共享 prompt 场景中，不同请求可能重复执行相同前缀的 prefill；在跨 worker 或跨节点 serving 中，KV 的传输、迁移、复用和远程访问会进入端到端时延关键路径。相应地，PagedAttention、prefix cache、KV compression、KV quantization、KV eviction、KV offloading、distributed KV cache 和 remote KV serving 等方法，都是围绕“如何降低状态成本、提高状态复用、控制状态移动”展开。

2.2.3 请求动态性与调度问题

LLM 请求具有高度动态性：prompt 长度不同、输出长度未知、请求到达时间随机、服务质量目标不同。Prefill 和 decode 的资源特征也不同，长 prefill 可能阻塞短 decode，请求完成时间差异又会导致 batch 内部动态变化。

调度问题的核心是吞吐和时延之间的动态权衡。等待凑 batch 可以提高硬件利用率，但会增加 TTFT；过大的 batch 会推高单请求延迟和显存占用；长 prompt prefill 可能阻塞已经进入流式输出的 decode 请求；不同租户、优先级和 SLA 又会使简单 FIFO 策略失效。因此，现代 LLM serving 通常采用 dynamic batching、continuous batching、iteration-level scheduling、chunked prefill、deadline-aware scheduling、priority scheduling 和 admission control 等机制，在 batch 形成、active set 更新和请求优先级之间做在线决策。

2.2.4 Decode 串行性与生成加速问题

LLM decode 是自回归过程，每一步依赖前一步输出，天然存在串行依赖。即使单步 forward 被加速，逐 token 生成仍可能造成较高 TPOT 和 ITL。

这一瓶颈与模型规模无关，即使单步 decode 的 kernel 足够快，自回归依赖仍会限制输出速度。单步 decode 的计算粒度较小，硬件利用率往往不如 prefill；sampling、beam search 或复杂 decoding policy 还会增加控制开销。Speculative decoding、assisted decoding、parallel decoding、tree decoding、beam search optimization、multi-token prediction 和 adaptive drafter selection 等方法，试图通过草稿生成、批量验证或多 token 预测减少目标模型的串行调用次数，但它们也必须在 acceptance rate、额外计算、输出质量和系统复杂度之间取得平衡。

2.2.5 多实例、多 GPU 与服务级编排问题

当模型规模、并发量或服务可用性需求超过单设备能力时，系统需要跨多个 GPU、worker、replica 或节点协同。此时问题从单个 engine 内部执行扩展为分布式执行和服务级调度。

分布式化带来的收益与代价同时存在。Tensor、pipeline、expert 或 context parallel 可以突破单设备容量和吞吐限制，但会引入 activation、KV 和中间状态的跨设备传输；prefill 与 decode 的资源特征差异使 disaggregation 成为可能，也使 KV handoff 成为新的关键路径；多 replica 和多模型服务可以提高可用性和弹性，却需要在负载均衡、质量、时延和成本之间做服务级选择。因此，distributed parallelism、prefill/decode disaggregation、distributed KV cache、KV transfer、replica routing、cluster load balancing、cascade serving 和 speculative serving orchestration 等方法，本质上都在处理“如何将单实例推理扩展成可控的分布式服务”这一问题。

2.3 LLM Inference Optimization Taxonomy: 以优化对象为主轴的分类

围绕 LLM inference 相关优化方向分类的 overview 和 survey 工作较多，表 4 汇总了其中代表性工作的分类方式及其特点。

表 4 现有 LLM Inference Overview 与 Survey 的分类方式

工作	分类视角	主要分类	分类特点
<i>A Survey on Efficient Inference for Large Language Models</i> [73]	优化层次	Data-Level、Model-Level、System-Level	覆盖从输入数据、模型结构到系统实现的多层优化，适合从整体上理解推理效率优化
<i>A Survey of LLM Inference Systems</i> [75]	系统流水线与系统形态	Request processing、model optimization and execution、memory management、以及 single-replica、multi-replica、disaggregated、serverless systems	对 system-level 方法拆分较细，突出 kernel、batching/scheduling、memory management 与不同部署形态
<i>Taming the Titans: A Survey of Efficient LLM Inference Serving</i> [74]	Serving 层级与场景	Instance-Level Approaches、Cluster-Level Strategies、Emerging Scenarios、Miscellaneous Areas	强调从单实例优化扩展到集群部署、负载均衡和云服务等 serving 问题

基于这些工作，本报告进一步整合形成 LLM 推理优化方向的 **Layer-Scope Taxonomy**：以主要优化对象所在的系统抽象层为主轴，将方法分为 Model-Level、Kernel-Level、Runtime-Level 和 Serving Policy & Routing-Level；同时以执行范围为副轴，区分 Local 与 Distributed。后续 2.3.1–2.3.4 将依次介绍四个层级，并在每个层级中讨论其本地与分布式实现。

其中，Local 表示优化限制在单个 serving instance 或本地控制域内，可以包含 CPU-GPU/NPU 和本地存储等异构资源协同；Distributed 表示优化需要跨多个 GPU、device、worker、replica、service 或 node 协同，并涉及 computation、request、KV state、activation 或 routing decision 的跨实体分布。需要说明的是，这一分类并不要求所有方法完全互斥，真实 LLM serving 系统通常具有 cross-layer co-design 特征。因此，在归类一项具体工作时，本报告主要考察三个问题：它解决什么 inference problem，主要优化对象位于哪一层，以及该优化发生在 Local 还是 Distributed scope。

2.3.1 Model-Level Optimization

Model-Level Optimization 的核心是改变模型本身，从源头降低 inference complexity。它优化的是模型参数、参数表示、结构设计或算法语义。只要优化后的模型即使脱离具体 serving engine 仍然成立，就可以认为主要属于 Model-Level。这一层的优化对象包括参数量、权重表示、FLOPs、activation 计算量、attention complexity、KV size 以及条件计算路径。

Local scope: 常见方法包括 weight quantization、pruning、distillation、low-rank、weight sharing、GQA/MQA/MLA、linear 或 sparse attention、state space model、early exit、token pruning 和 dynamic sparsity 等，主要服务于单实例或单设备上的模型压缩和结构简化。

Distributed scope: Model-Level 更强调模型结构是否适合跨设备执行，例如面向 expert parallelism 的 MoE architecture、更适合 tensor/context parallel 的 attention 或 KV 结构、更容易按 layer、expert、context 或 activation 切分的模型结构，以及由 small / medium / large model 组成的多层级 model family。

需要注意，Model-Level 不包括一般的 execution graph optimization。CUDA Graph、operator fusion、TensorRT execution graph 优化等主要改变的是执行路径或部署图，而不是模型语义本身，应归入 Runtime-Level 或 Kernel-Level。

2.3.2 Kernel-Level Optimization

Kernel-Level Optimization 的核心是在不改变模型数学语义的前提下，优化 tensor operator 或 communication primitive 在硬件上的执行效率。它关注的是 attention、GEMM、normalization、sampling、collective communication 等底层执行单元如何更快、更省显存带宽、更高效地利用硬件，其直接目标通常是提高 GPU/NPU utilization、降低 HBM traffic、增强 SRAM/cache reuse、提升 Tensor Core 或 NPU special unit 利用率，并改善 operator throughput。

Local scope: 典型方法包括 FlashAttention、FlashDecoding、paged attention kernel path、fused MLP、fused RMSNorm、fused softmax、low-bit GEMM、FP8/INT8/INT4 kernels、hardware-aware tiling、memory layout 优化和 GPU-side sampling 等。

Distributed scope: Kernel-Level 主要体现为 collective 或 P2P communication primitive 的实现优化，例如 all-reduce、all-gather、reduce-scatter、all-to-all、RDMA/NVLink transfer path、fused communication primitive、collective algorithm selection 以及 MoE expert dispatch / combine primitive。

需要区分的是，如果优化对象是 collective / P2P primitive 的实现路径，归为 Kernel-Level $\times$ Distributed；如果优化对象是通信与计算在 runtime pipeline 中的编排、重叠和调度，则归为 Runtime-Level $\times$ Distributed。

2.3.3 Runtime-Level Optimization

Runtime-Level Optimization 是现代 LLM serving 的核心层。它发生在请求进入 inference engine 之后，围绕在线 token execution、KV state、batch composition、decode loop 和 execution pipeline 做动态管理。Runtime-Level 不改变模型本身，也不一定改变单个 kernel，而是决定请求到达后如何高效执行。Runtime-Level 可以进一步细分为四类：KV / Memory Runtime、Batching / Scheduling Runtime、Decoding Runtime 和 Execution Runtime。

A. KV / Memory Runtime KV / Memory Runtime 关注 KV lifecycle、memory state 和 cache reuse。它的核心问题是：KV Cache 如何存储、复用、压缩、淘汰、迁移或远程访问。

Local scope: 主要包括 KV cache management、PagedAttention / memory paging、Prefix cache / RadixAttention、KV compression / quantization、KV eviction、KV offloading、cache persistence 和 runtime memory allocator 等机制。

Distributed scope: 问题进一步扩展为 distributed KV cache、KV transfer / migration、remote KV serving、multi-replica KV reuse、distributed prefix cache、KV-aware placement 和 cross-worker state synchronization。

这一类优化的核心目标是降低 KV 显存占用、减少重复 prefill、提升 cache reuse，并在分布式场景下控制 KV 传输和状态迁移成本。

B. Batching / Scheduling Runtime Batching / Scheduling Runtime 关注请求和 token 在 inference engine 内如何组织执行。它回答的是：哪些请求何时执行，哪些 token 可以组成 batch，prefill 和 decode 如何交错，如何在 throughput、TTFT、TPOT、ITL、公平性和 SLA 之间做权衡。

Local scope: 典型机制包括 dynamic batching、continuous batching、iteration-level scheduling、chunked prefill、prefill/decode interleaving、engine-local admission control、priority / fair scheduling 和 deadline-aware scheduling。

Distributed scope: 调度对象会跨越 worker、replica 或资源池，因此需要 cross-worker scheduling、distributed continuous batching、prefill worker / decode worker scheduling、heterogeneous GPU scheduling、multi-replica queue coordination、distributed admission control 和 SLO-aware worker scheduling。

这一类优化的核心目标是在动态请求负载下提高硬件利用率，同时控制排队时延和服务质量。

C. Decoding Runtime Decoding Runtime 关注在线 token generation procedure。它回答的是：输出 token 如何生成、验证、选择和接受。它不主要决定请求如何组 batch，也不主要决定计算如何 launch，而是改变 decode loop 的生成逻辑。

Local scope: 这类方法包括 speculative decoding、assisted decoding、parallel decoding、tree decoding、beam search optimization、n-gram speculation、prompt lookup decoding 和 serving-time multi-token prediction / verification。

Distributed scope: Decoding Runtime 会涉及 distributed verification、draft / target execution split、multi-draft execution、batched verification across workers、draft model 与 target model 分布式协同，以及 speculative decoding 与 prefill/decode disaggregation 的结合。

Speculative decoding 这类方法有时也被称为 decoding algorithm。本报告将其归入 Runtime-Level，是因为其主要改变 serving 时的在线 token generation procedure，而不是模型参数或架构本身。但如果某种 multi-token prediction 方法引入额外训练 heads 或改变模型结构，则应标注 Model-Level primary 或 secondary。

D. Execution Runtime Execution Runtime 关注 scheduled work 如何 launch、overlap、replay 和执行。当前面的 runtime 已经决定哪些请求要运行、batch 如何组成、decode 如何进行之后，Execution Runtime

负责将计算、数据搬运和通信高效提交到硬件，并组织成流水线执行。

Local scope: 主要包括 CUDA Graph / HIP Graph replay、async execution、stream scheduling、CPU-GPU overlap、host-device communication hiding、compute / memory-copy overlap、runtime metadata / buffer reuse 和 GPU-side sampling execution path。

Distributed scope: 主要包括 tensor parallel runtime、pipeline parallel runtime、expert parallel runtime、context / sequence parallel runtime、prefill/decode disaggregation、distributed execution pipeline、communication-computation overlap scheduling 以及 activation / KV transfer orchestration。

Execution Runtime 与 Kernel-Level 的边界在于：Kernel-Level 优化单个 operator 或 communication primitive 的实现；Execution Runtime 优化一组 operator、copy 和 communication 的 launch、同步、overlap 和流水化组织方式。

2.3.4 Serving Policy & Routing-Level Optimization

Serving Policy & Routing-Level Optimization 关注更粗粒度的服务级控制决策。它不直接优化单个 engine 内部 token 如何执行，而是决定请求进入哪个模型、adapter、replica、worker pool 或 service path，以及是否进行 fallback、cascade、escalation 或多服务编排。这一层的典型优化对象包括 request assignment、model / adapter selection、replica / worker selection、service path selection、fallback / escalation、cascade policy、SLO / cost / latency policy 和 multi-service orchestration。

Local scope: 主要包括 local model selection、adapter / LoRA routing、local cascade serving、normal decoding 与 speculative decoding path selection、draft selection、local/cloud fallback、budget-aware local inference 和 latency-aware local policy。

Distributed scope: 进一步扩展为 replica routing、worker-pool routing、cluster load balancing、SLO-aware routing、cost-aware routing、region-aware routing、KV-aware routing、speculative serving orchestration、draft-target worker orchestration 和 multi-service graph orchestration。

Runtime-Level 和 Serving Policy & Routing-Level 在真实系统中强耦合，但 primary decision object 不同。Runtime-Level 解决“请求进入某个 engine 后如何执行”；Serving Policy & Routing-Level 解决“请求应该交给哪个模型、worker、replica 或服务路径”。后者经常依赖 runtime feedback，例如 queue length、KV cache pressure、batch state 和 SLO slack。

2.4 工业级 LLM Inference Framework 代表性机制映射

为了避免分类停留在抽象概念层面，表 5 选取若干主流工业级 LLM inference framework 中最具有代表性的机制，并将其映射到前述 Layer-Scope Taxonomy。该表不追求覆盖各框架的全部功能，而是展示每个框架最核心、最能体现其系统设计特点的若干机制。

表 5 工业级 LLM Inference Framework 代表性机制映射

Framework	代表性方法 / 机制	解决的具体问题	优化方向分类层 & Scope	具体方法说明
vLLM [5]	PagedAttention	KV Cache 动态分配和显存碎片问题	Runtime / KV-Memory $\times$ Local	通过分页式 KV 管理降低显存碎片，提高 KV Cache 利用率
	Continuous batching	Decode 阶段请求动态进出，静态 batch 利用率低	Runtime / Batching-Scheduling $\times$ Local	在 decode iteration 动态加入和移除请求，提高高并发吞吐
	Chunked prefill	长 prompt prefill 阻塞 decode，影响 TTFT / ITL	Runtime / Batching-Scheduling $\times$ Local	将长 prefill 拆分为 chunk，与 decode 交错执行
	Prefix caching	多请求共享前缀导致重复 prefill	Runtime / KV-Memory $\times$ Local/Distributed	复用共享 prompt 或 prefix，减少重复 prefill 计算
TensorRT-LLM [9]	In-flight batching	高并发请求长度不同，传统 batching 效率低	Runtime / Batching-Scheduling $\times$ Local	对不同阶段和长度的请求进行动态批处理
	Paged KV Cache	KV Cache 分配、复用和显存管理问题	Runtime / KV-Memory $\times$ Local	通过 paged KV 管理改善 KV Cache 利用率
	FP8 / INT8 / INT4 execution	权重、activation 和算子执行成本高	Model + Kernel $\times$ Local/Distributed	通过低比特模型表示和低比特 kernel 降低显存和计算成本
	Overlap scheduler	compute、copy、communication、prefill/decode 没有充分重叠	Runtime / Execution $\times$ Local/Distributed	将计算、通信和数据搬运流水线化，减少等待
SGLang [8]	RadixAttention	多轮、多分支请求存在大量共享前缀	Runtime / KV-Memory $\times$ Local	用 radix tree 组织 prefix cache，提高共享前缀复用
	Zero-overhead CPU scheduler	高并发场景下 CPU 调度开销影响吞吐	Runtime / Batching-Scheduling $\times$ Local	降低 CPU 侧调度和 metadata 管理开销
	Prefill-decode disaggregation	Prefill 和 decode 资源特征不同，混部互相干扰	Runtime / Execution $\times$ Distributed	将 prefill 与 decode 分离到不同资源池，减少资源干扰
	Multi-LoRA batching	多 adapter 请求混合服务时 batch composition 复杂，吞吐下降	Serving Policy & Routing + Runtime/Scheduling $\times$ Local/Distributed	将不同 LoRA / adapter 请求合并调度，提高多 LoRA serving 效率
Hugging Face TGI [11]	Continuous batching	incoming requests 动态变化，传统 batching 吞吐不足	Runtime / Batching-Scheduling $\times$ Local	动态合并请求，提高在线 serving 吞吐
	Flash Attention	attention HBM traffic 高，operator throughput 受限	Kernel $\times$ Local	通过高效 attention kernel 降低显存访问并提升算子吞吐
	Paged Attention	KV Cache / attention memory 管理效率不足	Runtime / KV-Memory $\times$ Local	通过 paged attention 改善 attention/KV memory 管理
	Tensor parallelism	单 GPU 容量或吞吐不足，需要跨 GPU 执行	Runtime / Execution $\times$ Distributed	将模型执行切分到多个 GPU，提高模型容量和吞吐
LMDeploy / TurboMind [70]	Persistent batching	高并发请求动态调度，提高吞吐	Runtime / Batching-Scheduling $\times$ Local	通过持续批处理机制维持 batch 执行效率
	Blocked KV Cache	KV Cache 显存管理和访问效率问题	Runtime / KV-Memory $\times$ Local	通过 block-based KV 管理改善显存利用率
	Dynamic split & fuse	长 prompt 和 decode 混合执行效率低	Runtime / Batching-Scheduling $\times$ Local	动态拆分和融合 prefill / decode 任务，提高混合负载效率
	Tensor parallelism	大模型需要跨 GPU 执行	Runtime / Execution $\times$ Distributed	支持模型在多 GPU 上并行执行

2.5 小结

本章首先介绍了 LLM inference 的基本执行机制，包括 prefill/decode、KV Cache、batching/scheduling 和主要性能指标。随后，从执行机制出发，总结了 LLM inference 的主要问题空间，包括计算复杂度与模型规模、KV Cache 与内存状态、请求动态性与调度、decode 串行性与生成加速，以及多实例、多 GPU 与服务级编排问题。

在已有 overview 和 survey 工作的基础上，本章进一步整合形成 Layer-Scope Taxonomy：以 Model、Kernel、Runtime、Serving Policy & Routing 作为优化对象主轴，以 Local / Distributed 作为执行范围副轴，并在 Runtime-Level 内部细分为 KV / Memory Runtime、Batching / Scheduling Runtime、Decoding Runtime 和 Execution Runtime。该分类方式能够同时描述优化机制所处的系统层级及其执行范围，并适用于分析具有 cross-layer co-design 特征的 LLM 推理系统。

最后，本章通过主流工业级 inference framework 的代表性机制映射，展示了真实系统通常不是单点优化，而是组合模型表示、kernel、KV/memory runtime、batching/scheduling、execution runtime、distributed execution 和 serving policy 等多层机制。该分类框架将作为后续分析 CEC/MEC 场景下模型部署、请求调度、KV 状态迁移、服务路径选择和端-边-云资源协同优化的基础。

三、CEC 场景下的大模型推理

3.1 CEC LLM 推理的系统形态

CEC 场景下的大模型推理面向端侧设备、接入边缘、区域边缘和中心云组成的多层协同系统。请求可以从终端、基站、边缘网关或区域入口进入，模型权重、adapter、KV Cache、prefix cache、session state 和中间状态也可能分布在不同层级。一次推理既可能在单个端侧或边缘实例内完成，也可能经过端-边、边-云或端-边-云联合执行。

与云侧集中式 LLM serving 相比，CEC 的根本变化在于系统边界被拉长了。云侧 serving 通常假设资源相对集中、互联稳定、控制面统一，优化重点是单实例或集群内部的吞吐、显存和调度效率。CEC 则把地理位置、无线链路、异构硬件、移动接入、状态分布和隐私边界都纳入推理路径。此时，推理系统不只是选择一个 GPU 实例执行请求，而是要决定请求在哪一层服务、模型如何部署、状态如何保留、网络传输是否值得、以及服务路径何时调整。图 3 展示了模型分割部署、计算任务调度和 KV / runtime state 调度在端、边、云多层环境中的协同关系。

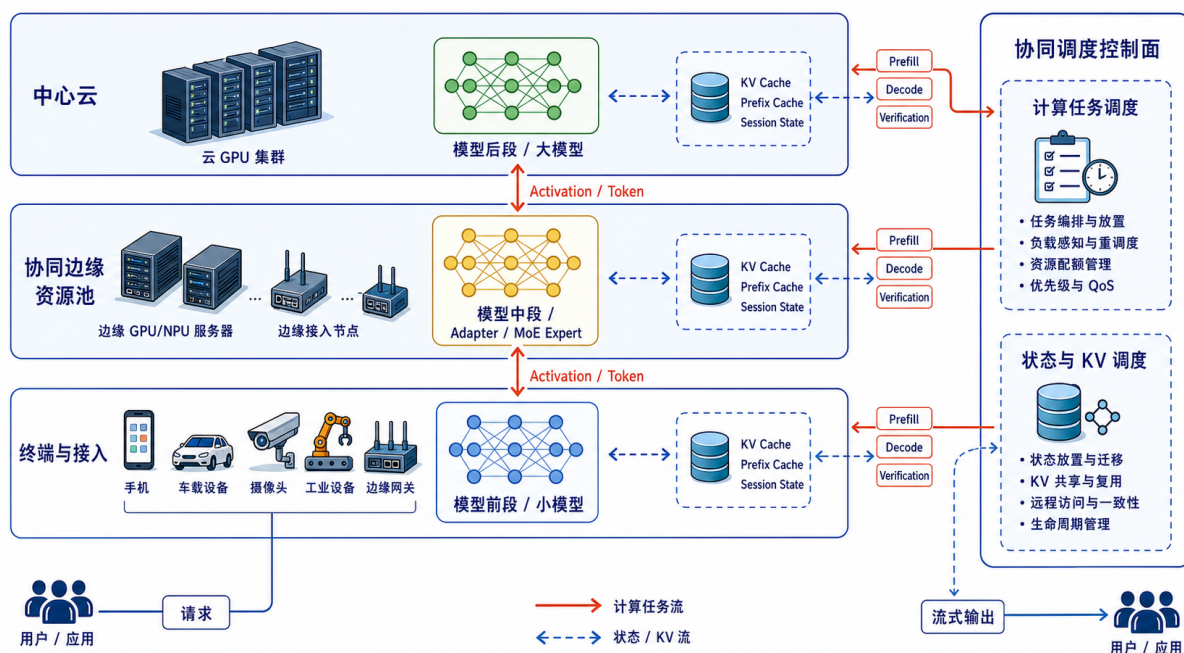


图 3 CEC 环境下的 LLM 推理 Serving 系统形态。Serving Policy 根据节点资源、链路与运行状态，协调端侧、接入边缘、区域边缘和中心云之间的模型分割与部署、Prefill/Decode 等计算任务调度，以及 KV Cache、Prefix Cache 和 Session State 的复用、迁移与远程访问。

因此，CEC LLM inference 可以看作是三类决策的叠加。第一类是空间部署决策，即模型层、张量、专家组件、推理阶段或生成职责如何放在端、边、云不同资源上。第二类是时间调度决策，即请求、token、prefill/decode 阶段和 active batch 如何在动态负载下组织执行。第三类是服务路径决策，即完整请求、会话、workflow 和运行时状态如何在多层网络中路由、卸载、迁移或复用。后续章节

的三条主线正是从这三个角度展开。

3.2 CEC 相较云侧集中式 LLM Serving 新增的问题空间

资源受限与硬件异构：云侧集中式 serving 的核心瓶颈通常来自模型规模、KV Cache、batching、并行执行和多实例调度。CEC 并没有消除这些问题，而是把它们放到一个资源更加受限、硬件更加异构的物理环境中。端侧和边缘节点的算力、显存、内存、功耗和散热预算有限，同一个模型未必能完整放入目标设备；即便能够部署，也可能受到低比特 kernel 支持、operator coverage、runtime backend 和 layout conversion 的限制。由此，模型部署问题从“如何在—组相似 GPU 上高效执行”，转变为“如何在能力差异显著的端-边-云节点之间选择模型形态和执行拓扑”。

网络波动与状态移动成本：端-边、边-边和边-云链路的带宽、RTT、抖动和拥塞状态会直接影响模型切分、task offloading、prefill/decode 分离和 KV 迁移是否值得执行。对于普通 DNN 推理，跨节点传输的主要对象通常是输入或中间 activation；而在 LLM inference 中，KV Cache、prefix cache、session state 和多轮上下文也会成为需要迁移或远程访问的状态资源。网络成本因此不再只是请求入口到服务实例的传输延迟，而是贯穿推理全过程的状态移动成本。

低并发、突发请求与调度失配：单个接入边缘节点往往并发规模有限，请求到达具有地理局部性和突发性，简单等待大 batch 会显著增加 TTFT；但如果完全放弃 batching，又会降低 GPU/NPU 利用率并抬高单位 token 成本。与此同时，prefill 和 decode 的资源画像差异、长 prompt 和短 prompt 的混合、不同业务的 SLA 差异，都会使固定 batch size 和 FIFO 队列难以稳定工作。边缘 runtime 因此更需要短窗口、SLO-aware、phase-aware 和状态感知的在线调度。

用户移动与服务连续性：对于单轮短请求，移动可能只改变入口节点；对于多轮对话、长上下文和 agent workflow，移动会改变服务实例与状态位置之间的关系。KV Cache、prefix cache、session state 或 agent memory 可能仍然停留在旧边缘节点，新请求却已经从新入口进入系统。系统此时需要在迁移状态、远程访问旧状态、重新 prefill 或降低上下文质量之间做选择。这个问题使 CEC LLM inference 的优化目标从单次请求时延扩展到会话连续性、状态复用和移动场景下的 QoS 稳定性。

隐私边界与数据本地性：部分 prompt、传感器数据、业务上下文或用户状态可能不能上传到云端；某些行业应用还要求推理过程中的中间状态不离开边缘域。隐私约束会限制模型选择、offloading 范围、KV 迁移路径和 fallback 策略，使系统不能仅按最小时延或最低成本选择节点。换言之，CEC 中的推理优化必须同时处理资源可行性、链路代价、状态位置、隐私边界和业务 QoS。

3.3 CEC 场景下 LLM Inference 优化重心的转移

Model-Level：优化重心从单模型压缩转向分层模型体系与可切分性。前文提出的 Layer-Scope taxonomy 仍然适用于 CEC 场景，但每一层的关注点都会发生迁移。Model-Level 不再只是压缩模型或降

低 FLOPs，而是要形成端侧小模型、边缘中等模型、云端大模型及其 adapter / MoE / 分层模型族之间的协同体系。模型结构是否容易按 layer、tensor、expert、stage 或 semantic responsibility 切分，会直接影响后续部署和 offloading 的可行性。

Kernel-Level: 优化重心从单一 GPU 性能转向异构硬件适配与稳定执行。CEC 系统可能同时包含端侧 NPU、移动 GPU、边缘 GPU/NPU、CPU 和云端 GPU，不同后端对 precision、operator、attention kernel、KV layout 和通信 primitive 的支持并不一致。一个模型或 runtime 在云端高效，并不意味着它能在边缘设备上稳定运行；unsupported operator、layout conversion 和 fallback 甚至可能抵消协同执行带来的收益。

Runtime-Level: 优化重心从实例内部执行转向网络、计算与状态的联合在线控制。云侧 runtime 主要管理单个 serving engine 或 GPU cluster 内的 KV、batch、decode loop 和 execution pipeline；CEC runtime 则必须把网络状态、节点负载、KV 位置、用户移动和业务 SLA 纳入同一个在线控制面。KV / Memory Runtime 从单实例显存管理扩展为分层状态管理；Batching / Scheduling Runtime 从大请求池吞吐优化转向低并发、短等待、SLO-aware 和 phase-aware 调度；Decoding Runtime 需要考虑端侧或边缘小模型作为 drafter 的可用性；Execution Runtime 则要把端侧执行、网络传输和边缘/云端计算组织成可重配置的流水线。

Serving Policy & Routing-Level: 优化重心从副本选择转向端-边-云服务路径的联合决策。这一层需要联合考虑模型层级、任务 offloading、状态位置和隐私约束。一个请求是否应该本地执行、卸载到接入边缘、升级到区域边缘或转发到云端，取决于请求复杂度、模型能力、链路状态、节点负载、KV/cache 位置、用户移动趋势和 SLA 风险。该层还需要处理 fallback、escalation、state-aware routing 和 mobility-aware routing，使服务策略能够随着环境变化而调整。

总体变化: Distributed Scope 被显著强化，各层之间的耦合更加紧密。CEC 并不是在 LLM inference taxonomy 之外新增一个孤立层，而是显著强化了 Distributed Scope 和 Serving Policy & Routing-Level 的作用，并使 Model、Runtime 和 Routing 之间的边界更紧密。模型是否可切分会影响部署空间，batching 是否可调整会影响请求 offloading 收益，KV 是否可迁移会影响服务路径选择。后续章节不再按单一抽象层展开，而是围绕最能体现 CEC 特征的三类算法问题组织相关工作。

3.4 本文采用的 CEC LLM Inference 综述框架

基于上述分析，本文将 CEC 场景下的 LLM inference 优化归纳为三条主线。第一条是资源自适应模型部署方法，它对应空间部署问题，关注模型结构、模型组件、推理阶段和生成职责如何根据端、边、云资源能力进行切分、放置和动态调整。该方向覆盖 layer-wise / vertical partitioning、tensor / operator-level partitioning、expert / component-level partitioning、inference phase partitioning 和 semantic / token-level partitioning，第 4 章将围绕这一类方法展开。